

Computer Vision Explorer · Full Curriculum

2-session series (2 hrs each) · 11-15 yrs · Intermediate → Advanced

Full facilitator curriculum **PREMIUM**

Overview

Build on the free Teachable Machine session to make a working computer-vision app.

DETAIL

Track	AI Skills for Kids
Format	2-session series (2 hrs each)
Ages	11-15 yrs
Level	Intermediate → Advanced
Price	from KES 7,000
Standalone	No
Prerequisites	Teachable Machine (free)

Course Overview

Computer Vision Explorer is the intermediate follow-on to TinkerWithMe's free Teachable Machine taster session. Where the free session shows learners that a computer can be trained to "see," this two-session series pushes them to build a genuinely useful, demoable application powered by their own trained model. Learners move from casual experimentation into disciplined model-building: collecting deliberate, varied training data, testing for bias and failure cases, and then wiring a trained classifier into a real webpage that responds live to a camera feed.

The arc across the two sessions is: **Session 1** builds the brain (a robust multi-class image classifier for waste sorting, plant identification, or hand gestures, trained with attention to data quality and bias), and **Session 2** builds the body (a working

web app — a gesture-controlled slide advancer or a waste-sorting advisor — that uses that trained model live through a webcam). Learners write and edit real JavaScript, not just clicking through a no-code tool.

By the end, every learner (or pair) leaves with a working computer-vision web app running on their own laptop that they can demo to family, friends, or at a school science fair — plus the exported model link, the full source code, and a one-page "how it works" explainer they write themselves.

Learning Outcomes

- Design a multi-class image classification scheme (choosing classes that are distinct, balanced, and useful) and justify the choice.
- Collect and curate training data deliberately — varying background, lighting, angle, and distance — to reduce overfitting and bias.
- Explain, with concrete examples, how biased or low-quality training data produces biased or unreliable model behaviour.
- Train, evaluate, and iterate on an image classifier using Teachable Machine, reading confidence scores and diagnosing misclassifications.
- Export a trained model and connect it to a live webcam feed inside a real webpage using HTML, JavaScript, and the ml5.js library.
- Read, modify, and debug JavaScript that reacts to model predictions in real time (thresholds, cooldown timers, conditional logic).
- Demo and explain their own computer-vision application clearly to a non-technical audience.

Materials Master List

ITEM	SPEC	QTY PER STUDENT/ GROUP	NOTES
Laptop with webcam	Windows/Mac/Chromebook, Chrome or Edge browser installed, camera permissions enabled	1 per 2 students	Test camera + browser permissions before session starts
Internet connection	Reliable Wi-Fi, min 10 Mbps shared	1 for room	Teachable Machine and ml5.js CDN both require

live internet; bring a mobile hotspot as backup

Code editor	Visual Studio Code, with "Live Server" extension pre-installed	1 install per laptop	Install and test the Live Server extension before Session 2
Starter project files	Facilitator-prepared folder containing blank <i>index.html</i> template and this session's two working app files	1 folder per laptop, pre-loaded	See Session 2 build detail for exact file contents
Projector/screen + HDMI or wireless casting	Any classroom projector	1 for room	Used for facilitator live demo and final showcase
Whiteboard + markers	Standard	1 for room	For class-planning and bias discussion diagrams
Waste-sorting realia set	6-8 clean items per group: plastic bottle, cardboard scrap, banana peel/ fruit scrap (or printed photo), can, paper cup, food wrapper, glass jar, plastic bag	1 set per group of 3-4	For groups choosing the waste-sorting track; wash/ dry items in advance
Labelled mini-bins or boxes	3 small boxes labelled Recyclable (blue), Compostable (green), Landfill (black)	1 set per group	Optional physical prop for showcase demo
Plant/leaf reference cards	Printed photos or real leaf samples of 3-4 distinct local plant species	1 set per group choosing plant track	Laminate for reuse across cohorts
Gesture reference poster	A4 printout showing 3 example hand gestures (wave left, wave right, neutral/background)	1 per group choosing gesture track	Pin near each station as a reminder during data collection
Bias-reflection worksheet	1-page printed worksheet (facilitator prints from this curriculum's Session 1 discussion notes)	1 per student	Filled in during Session 1 bias discussion
Sticky notes + pens	Standard	4-5 per student	Used for class-brainstorm and showcase peer feedback
Power strips/ extension cords	4-6 outlets each	2-3 per room	Laptops will run 4 hours total across two sessions
USB flash drive	8GB+	1 for facilitator	Backup way to move starter files/model links if cloud access fails
		1 per student	See Session 2 build detail

Session Plans

Session 1: Advanced Classifiers

Objectives

- Choose and scope a multi-class image classification task (waste sorting, plant ID, or hand gestures).
- Collect deliberately varied training data and train a model in Teachable Machine to a usable accuracy.
- Identify at least one concrete example of bias or data-quality weakness in their own model and describe how to fix it.

Timing (120 minutes)

- 0:00-0:10 — Welcome back; recap free session (what Teachable Machine is, how training works)
- 0:10-0:25 — Mini-lecture + discussion: training-data quality and bias, with live examples
- 0:25-0:40 — Groups form (3-4 students); choose track (waste / plant / gesture); define exact class list
- 0:40-1:10 — Data collection: capture 150-200 images per class with deliberate variation
- 1:10-1:30 — Train model in Teachable Machine; read accuracy/confusion feedback in Preview panel
- 1:30-1:45 — Bias stress-test: test model against new conditions, log failures
- 1:45-1:55 — Targeted retraining: add data to fix the weakest class
- 1:55-2:05 — Export model, save shareable link, complete bias-reflection worksheet
- 2:05-2:10 — Wrap-up: preview Session 2 (what we'll build with this model)

Step-by-step

1. Open with a show of hands: "Who tested their Teachable Machine model at home and it got something wrong?" Use 1–2 stories to bridge into today's theme: *garbage in, garbage out* — a model is only as good as its training data.
2. Run the bias mini-lecture (see Build detail below) using the projector. Keep it to 15 minutes; this is discussion-led, not lecture-heavy.
3. Split the class into groups of 3–4. Each group picks ONE track: **Waste Sorting** (recyclable / compostable / landfill), **Plant ID** (3–4 species from the reference cards), or **Hand Gestures** (wave_left / wave_right / neutral). Write each group's chosen classes on the whiteboard so the facilitator can track who needs which realia set.
4. Hand out realia sets, plant cards, or point groups to the gesture poster. Each group opens Teachable Machine (teachablemachine.withgoogle.com) → "Get Started" → "Image Project" → "Standard image model."
5. Walk the whole room through renaming classes to match their chosen labels exactly (lowercase, no spaces — e.g. `wave_left` not "Wave Left") — this matters because these exact strings will be used as code later.
6. Circulate constantly during data collection (30 minutes) — this is the highest-value coaching window. Push every group to vary background, lighting, distance, and angle for each class (see Build detail).
7. Bring the room back together briefly (2 minutes) before training: remind everyone to click "Train Model" and NOT close the browser tab while training runs.
8. While models train, walk around and help groups read the live Preview panel — point out confidence percentages and ask "would you trust this at 60%? At 95%?"
9. Lead the bias stress-test as a structured activity (see Build detail) — every group must find and record at least one real failure.
10. Give groups 10 minutes to add 20–30 more images specifically targeting their weakest class, then retrain.
11. Show the whole class how to click "Export Model" → "Upload my model" → copy the shareable link. Every group pastes their link into a shared document (Google Doc or Padlet) titled with their group name — this is what they'll use in Session 2.

12. Hand out the bias-reflection worksheet for the last 10 minutes; collect or photograph completed worksheets for the portfolio.

Build detail

Tool: Teachable Machine (teachablemachine.withgoogle.com), Chrome browser, no installation required, no account needed to train (a Google account is only needed if a group wants to save a project for later — the shareable model link works either way).

Bias mini-lecture content (use these exact examples):

- Show two mental scenarios: a waste-sorting model trained only on clean, well-lit bottles photographed against a plain white wall will likely fail on a crumpled, dirty bottle photographed on a cluttered classroom floor — because the training data never showed it variation.
- Show a gesture-recognition scenario: if every training image of "wave" was captured by the same person, in the same shirt, against the same wall, the model may have secretly learned "that shirt" or "that wall" rather than "waving," and will fail for a different student.
- Name the core lesson explicitly: **a model learns whatever pattern is common across its training images — including patterns you didn't intend it to learn.** This is real-world AI bias in miniature (e.g., facial recognition systems performing worse on skin tones underrepresented in training data).

Data collection workflow (exact steps students follow per class):

1. Click the webcam icon under the class name, click "Hold to Record," and capture at least 40–50 images in a first short burst.
2. Physically move: step left, step right, tilt the item/hand, move closer, move further away. Capture another 40–50 images.
3. Change the background: turn to face a different wall, add or remove classroom clutter behind the object. Capture another 30–40 images.
4. If lighting varies in the room (near a window vs. under a light), capture a final 20–30 images in the different lighting.
5. Target total: 150–200 images per class, and critically, an even number of images across all classes (a class with far fewer images will be predicted less often).
6. For every track, add a **"background" or "nothing" class** with 100+ images of the empty scene (empty hands, empty table, empty floor) — without it, the

model will be forced to guess a real class even when nothing relevant is in frame, which causes constant false triggers in Session 2's app.

Sample class lists to give groups exactly as a starting point:

- Waste sorting: `recyclable`, `compostable`, `landfill`, `background`
- Plant ID: `species_a`, `species_b`, `species_c`, `background` (rename to real species names from the reference cards)
- Gestures: `wave_left`, `wave_right`, `background`

Bias stress-test (structured 15-minute activity): After the first training run, each group must deliberately try to break their own model and record results in this exact format on a sticky note: "We tried ___ and the model said ___ with ___% confidence — this is a problem because ___." Required tests for every group: (1) test a class member who did NOT appear in training images, (2) test in a different corner of the room, (3) test with only the "background" class in frame (should not falsely trigger another class). Groups then choose ONE fix — usually adding 20–30 more varied images to the weakest class — and retrain, comparing before/after confidence.

What a good result looks like: After retraining, all classes score above ~85% confidence on the Preview panel when tested live, the "background" class correctly suppresses false positives when nothing relevant is in frame, and the group can articulate in one sentence what data gap they fixed.

Testing/iteration: Use the Teachable Machine Preview panel throughout — it classifies the live webcam feed in real time with a confidence bar per class, so learners get instant feedback without leaving the training tool.

Checks for understanding

- "Why did we add a 'background' class instead of just training three classes?"
- "If your model works perfectly for you but fails for your teammate, what's the most likely cause?"
- "What's one specific thing you changed between your first and second training run, and why?"

Common mistakes & fixes

MISTAKE	FIX
---------	-----

All training images captured from one fixed camera position/angle	Coach students to physically move the camera or object between captures; require at least 3 distinct angles per class
No "background/nothing" class trained	Add one with 100+ varied empty-scene images before moving on — this is non-negotiable, it breaks Session 2's app otherwise
Uneven image counts across classes (e.g. 200 vs. 40)	Point out the image count shown under each class name; top up the smallest class before training
Group closes the browser tab while "Training Model..." is running	Remind groups to wait ~1-2 minutes and watch the progress bar; if closed, they must retrain from scratch
Group can't find their exported model link for Session 2	Require every group to paste their shareable link into the shared class document before leaving — check this before dismissal

Session 2: Real-World Application

Objectives

- Connect a Teachable Machine model to a live webcam feed inside a real webpage using ml5.js.
- Build and customise a working gesture-controlled presentation OR waste-sorting advisor app end-to-end.
- Test the finished app for reliability (false triggers, confidence thresholds) and iterate on the code.

Timing (120 minutes)

- 0:00-0:10 — Recap Session 1 models; state today's goal (a real working app, not just a browser demo)
- 0:10-0:25 — Tour the starter files and explain the three moving parts: webcam, model, app logic
- 0:25-0:45 — Guided build: get the webcam + model talking and printing predictions live
- 0:45-1:10 — Branch by track: build gesture-controlled slides OR waste-sorting advisor, using the full provided code
- 1:10-1:35 — Personalise: swap in their own class names, edit slide text/waste tips, tune thresholds
- 1:35-1:50 — Test & iterate: deliberately break it, fix false triggers, tune cooldown/confidence values
- 1:50-2:05 — Polish + rehearse a 60-second demo pitch in groups

- 2:05-2:20 — Showcase: every group demos live; peer + facilitator feedback

Step-by-step

1. Recap: ask each group to say their track and one thing they fixed about their model last session.
2. Open VS Code with the pre-loaded starter folder on each laptop. Explain the three-part architecture on the whiteboard: *Webcam feed → ml5.js Image Classifier (using their exported model link) → JavaScript logic that reacts to predictions.*
3. Live-code on the projector: paste in the minimal webcam + model + `console.log(prediction)` skeleton first (see Build detail), run it with Live Server, and get the whole room to see live confidence percentages printing in the browser console/page before adding any app logic.
4. Have every group paste their own Session 1 model link into the `MODEL_URL` constant. Confirm predictions are showing their own class names, not a placeholder.
5. Split by track: hand the Gesture groups the full Gesture-Controlled Presentation code; hand the Waste/Plant groups the full Waste-Sorting Advisor code (plant groups reuse the advisor's card format, swapping in species facts instead of bin instructions).
6. Walk through the code together line-by-line at the projector for 10 minutes, focusing on: where the model URL goes, where the class name strings must match exactly, and where the threshold/cooldown numbers live.
7. Let groups personalise: gesture groups edit the `slides` array text; waste/plant groups edit the `binInfo`/species object text and colours.
8. Run the deliberate "break it" test: ask every group to try to make their app misfire (wave too fast, stand at a weird angle, show nothing) and tune `CONFIDENCE_THRESHOLD` and `COOLDOWN_MS` / `HOLD_TIME_MS` until it feels reliable.
9. Give a 5-minute warning before showcase; groups prepare a one-sentence "what this does" and a live demo sequence.
10. Run the showcase (see Assessment & Showcase section) and close with a group reflection circle: "What was the hardest bug to fix?"

Build detail

Tools: Visual Studio Code with the Live Server extension (right-click `index.html` → "Open with Live Server" — this runs the page on `localhost` so the browser will grant

camera permission; opening the file directly by double-click often blocks the webcam). ml5.js is loaded from a CDN inside the HTML, so no installation is needed beyond the internet connection. Each group needs their own Session 1 Teachable Machine shareable model link.

Step 1 — minimal skeleton (project every group builds first, to confirm the model connects):

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Model Test</title>
<script src="https://unpkg.com/ml5@0.12.2/dist/ml5.min.js"></script>
</head>
<body>
<video id="webcam" autoplay playsinline muted width="320" height="240"></video>
<div id="predictionLabel" style="font-size:1.5em;">Loading model...</div>

<script>
const MODEL_URL = "https://teachablemachine.withgoogle.com/models/PUT_YOUR_MODEL_ID_HERE/";
const video = document.getElementById('webcam');
const labelEl = document.getElementById('predictionLabel');
let classifier;

navigator.mediaDevices.getUserMedia({ video: true })
  .then(stream => { video.srcObject = stream; })
  .catch(err => { labelEl.innerText = "Camera error: " + err; });

window.addEventListener('load', () => {
  classifier = ml5.imageClassifier(MODEL_URL + "model.json", video, () => {
    labelEl.innerText = "Model ready!";
    classifyVideo();
  });
});

function classifyVideo(){
  classifier.classify(video, gotResult);
}

function gotResult(error, results){
  if (error){ console.error(error); return; }
  const top = results[0];
  labelEl.innerText = top.label + " (" + Math.round(top.confidence * 100) + "%)";
  classifyVideo();
}
</script>
</body>
</html>
```

Good result at this stage: the page shows "Model ready!" then updates continuously with the group's own class names and a confidence percentage as they move in front of the camera. If it shows a generic error, check that `MODEL_URL`

ends in a trailing slash and that Live Server (not double-click) was used to open the page.

Step 2 — Track A: Gesture-Controlled Presentation (full working app):

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Gesture-Controlled Presentation</title>
<script src="https://unpkg.com/ml5@0.12.2/dist/ml5.min.js"></script>
<style>
  body{font-family:Arial, sans-serif; text-align:center; background:#111; color:white; margin:0; padding:
0;}
  #webcam{position:fixed; bottom:16px; right:16px; width:180px; border:3px solid white; border-radius:8px;
z-index:10;}
  #predictionLabel{position:fixed; bottom:200px; right:16px; background:rgba(0,0,0,0.6); padding:6px 10px;
border-radius:6px; font-size:0.9em;}
  #slideDeck{display:flex; flex-direction:column; justify-content:center; align-items:center; height:
100vh;}
  #slideContent{max-width:700px; padding:0 20px;}
  #slideCounter{position:fixed; top:16px; left:16px; font-size:0.9em; opacity:0.7;}
</style>
</head>
<body>
<div id="slideCounter"></div>
<div id="slideDeck"><div id="slideContent"></div></div>
<video id="webcam" autoplay playsinline muted width="180" height="135"></video>
<div id="predictionLabel">Loading model...</div>

<script>
const MODEL_URL = "https://teachablemachine.withgoogle.com/models/PUT_YOUR_MODEL_ID_HERE/";
const CONFIDENCE_THRESHOLD = 0.85;
const COOLDOWN_MS = 1500;

const slides = [
  { title: "Welcome!", body: "Wave your RIGHT hand to begin." },
  { title: "What is Computer Vision?", body: "Teaching computers to recognise patterns in images and
video." },
  { title: "How We Trained It", body: "We collected hundreds of images of each gesture using Teachable
Machine." },
  { title: "Live Demo", body: "Wave right for next, wave left to go back – no mouse needed!" },
  { title: "Thank You", body: "Built by our team at TinkerWithMe Computer Vision Explorer." }
];

let currentSlide = 0;
let lastTriggerTime = 0;
const video = document.getElementById('webcam');
const labelEl = document.getElementById('predictionLabel');
let classifier;

navigator.mediaDevices.getUserMedia({ video: true })
  .then(stream => { video.srcObject = stream; })
  .catch(err => { labelEl.innerText = "Camera error: " + err; });

window.addEventListener('load', () => {
  classifier = ml5.imageClassifier(MODEL_URL + "model.json", video, () => {
    labelEl.innerText = "Model ready!";
```

```

        classifyVideo();
        renderSlide();
    });
});

function classifyVideo(){
    classifier.classify(video, gotResult);
}

function gotResult(error, results){
    if (error){ console.error(error); return; }
    const top = results[0];
    labelEl.innerText = top.label + " (" + Math.round(top.confidence * 100) + "%)";

    const now = Date.now();
    if (top.confidence >= CONFIDENCE_THRESHOLD && (now - lastTriggerTime) > COOLDOWN_MS){
        if (top.label === "wave_right"){ nextSlide(); lastTriggerTime = now; }
        else if (top.label === "wave_left"){ prevSlide(); lastTriggerTime = now; }
    }
    classifyVideo();
}

function nextSlide(){ currentSlide = Math.min(currentSlide + 1, slides.length - 1); renderSlide(); }
function prevSlide(){ currentSlide = Math.max(currentSlide - 1, 0); renderSlide(); }

function renderSlide(){
    const s = slides[currentSlide];
    document.getElementById('slideCounter').innerText = "Slide " + (currentSlide + 1) + " / " +
slides.length;
    document.getElementById('slideContent').innerHTML = "<h1>" + s.title + "</h1><p style='font-size:
1.3em;'>" + s.body + "</p>";
}
</script>
</body>
</html>

```

Exact edit instructions for gesture groups: replace `MODEL_URL` with their own link; confirm their two gesture class names exactly match `wave_right` and `wave_left` in the code (rename in code, not in the model, if they used different names); rewrite the five `slides` entries with their own presentation content.

Good result: standing in frame at a normal distance, waving right visibly advances the slide within about 1.5 seconds and does not double-trigger; waving left goes back; standing still or doing nothing does not change slides.

Step 2 — Track B: Waste-Sorting Advisor (full working app, also used by Plant-ID groups):

```

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Waste Sorting Advisor</title>

```

```

<script src="https://unpkg.com/ml5@0.12.2/dist/ml5.min.js"></script>
<style>
  body{font-family:Arial, sans-serif; text-align:center; background:#f4f4f4; margin:0; padding:20px;}
  #webcam{border:4px solid #333; border-radius:8px;}
  #card{margin:20px auto; max-width:400px; padding:20px; border-radius:12px; color:white; font-size:1.2em;
transition: background 0.3s;}
  #history{max-width:400px; margin:20px auto; text-align:left; background:white; padding:10px; border-
radius:8px;}
  .bin-recyclable{background:#1E88E5;}
  .bin-compostable{background:#43A047;}
  .bin-landfill{background:#424242;}
  .bin-background{background:#9E9E9E;}
  #confidenceBarOuter{background:#ddd; border-radius:6px; height:14px; margin-top:8px;}
  #confidenceBarInner{background:#FFD600; height:14px; border-radius:6px; width:0%;}
</style>
</head>
<body>
<h1>Waste Sorting Advisor</h1>
<video id="webcam" autoplay playsinline muted width="320" height="240"></video>
<div id="card" class="bin-background">
  <div id="label">Point the camera at an item...</div>
  <div id="confidenceBarOuter"><div id="confidenceBarInner"></div></div>
</div>
<div id="history"><strong>Recent scans:</strong><ul id="historyList"></ul></div>

<script>
const MODEL_URL = "https://teachablemachine.withgoogle.com/models/PUT_YOUR_MODEL_ID_HERE/";
const CONFIDENCE_THRESHOLD = 0.75;
const HOLD_TIME_MS = 1200;

const binInfo = {
  recyclable: { css: "bin-recyclable", instructions: "Rinse it and place it in the BLUE recyclables bin.",
fact: "Recycled plastic can be turned into new bottles or even clothing fibre!" },
  compostable: { css: "bin-compostable", instructions: "Place it in the GREEN compost bin.", fact: "Food
scraps can become nutrient-rich soil in as little as 6 weeks." },
  landfill: { css: "bin-landfill", instructions: "This goes in the BLACK general waste bin.", fact: "Many
landfill items take 100+ years to break down – try to spot fewer of these." },
  background: { css: "bin-background", instructions: "Show an item to the camera to get advice.", fact:
"" }
};

const video = document.getElementById('webcam');
const cardEl = document.getElementById('card');
const labelEl = document.getElementById('label');
const barEl = document.getElementById('confidenceBarInner');
const historyList = document.getElementById('historyList');

let classifier;
let stableLabel = null;
let stableSince = 0;
let lastLogged = null;

navigator.mediaDevices.getUserMedia({ video: true })
  .then(stream => { video.srcObject = stream; })
  .catch(err => { labelEl.innerText = "Camera error: " + err; });

window.addEventListener('load', () => {
  classifier = ml5.imageClassifier(MODEL_URL + "model.json", video, () => {
    labelEl.innerText = "Model ready – show an item!";
    classifyVideo();
  });

```

```

});
});

function classifyVideo(){
  classifier.classify(video, gotResult);
}

function gotResult(error, results){
  if (error){ console.error(error); return; }
  const top = results[0];
  barEl.style.width = Math.round(top.confidence * 100) + "%";

  if (top.confidence >= CONFIDENCE_THRESHOLD){
    if (stableLabel !== top.label){ stableLabel = top.label; stableSince = Date.now(); }
    if (Date.now() - stableSince >= HOLD_TIME_MS){
      showAdvice(top.label, top.confidence);
    }
  } else {
    stableLabel = null;
  }
  classifyVideo();
}

function showAdvice(classLabel, confidence){
  const info = binInfo[classLabel] || binInfo.background;
  cardEl.className = info.css;
  labelEl.innerHTML = "<strong>" + classLabel.toUpperCase() + "</strong> (" + Math.round(confidence * 100)
+ "%)<br>" + info.instructions + (info.fact ? "<br><em>" + info.fact + "</em>" : "");

  if (lastLogged !== classLabel){
    lastLogged = classLabel;
    const li = document.createElement('li');
    const time = new Date().toLocaleTimeString();
    li.innerText = time + " - " + classLabel;
    historyList.prepend(li);
    while (historyList.children.length > 5){ historyList.removeChild(historyList.lastChild); }
  }
}
</script>
</body>
</html>

```

Exact edit instructions: replace `MODEL_URL`; make sure the keys inside `binInfo` (`recyclable`, `compostable`, `landfill`, `background`) exactly match their Teachable Machine class names — if a group named their classes differently (e.g. `plastic`, `paper`) they must rename the keys in `binInfo` to match. Plant-ID groups relabel `binInfo` keys to species names and rewrite `instructions` / `fact` with real species care tips and fun facts instead of bin colours (they can also delete the bin CSS colour classes and just use one neutral card colour).

Good result: holding a real item steadily in frame for about a second produces a colour-coded card with correct sorting instructions and a fun fact; the history list

logs each new scan once (not repeatedly); showing nothing returns the neutral "point the camera" message.

Testing/iteration: Ask each group to test all their trained classes plus one "trick" test (an item they didn't train on, e.g. a random pencil) — a good app should either land on "background" or clearly hesitate at low confidence, not confidently misclassify. Tune `CONFIDENCE_THRESHOLD` up if there are false positives, or down if the app feels too hesitant. Tune `HOLD_TIME_MS` / `COOLDOWN_MS` up if triggers happen too easily, down if the app feels sluggish.

Checks for understanding

- "What are the three pieces of this app, and what does each one do?"
- "If your app keeps triggering the wrong slide/bin randomly, which two numbers in the code would you try changing first, and why?"
- "Why does the class name string in your code have to exactly match the class name in Teachable Machine?"

Common mistakes & fixes

MISTAKE	FIX
Opening <code>index.html</code> by double-click instead of Live Server	Browser blocks camera on <code>file://</code> pages in many setups; always right-click → "Open with Live Server"
Class name string in code doesn't match Teachable Machine class exactly (case, underscore, spelling)	Open the model's <code>metadata.json</code> in a browser tab (<code>MODEL_URL</code> + " <code>metadata.json</code> ") to see the exact labels array and copy them precisely
App triggers constantly even with nothing in frame	Confirm a "background" class was trained with enough images; raise <code>CONFIDENCE_THRESHOLD</code>
Slide/advice changes multiple times per single wave/hold	Increase <code>COOLDOWN_MS</code> (gesture app) or <code>HOLD_TIME_MS</code> (advisor app)
Page stuck on "Loading model..."	Check the browser console (F12) for a 404 — usually the <code>MODEL_URL</code> is missing the trailing slash or was mistyped

Assessment & Showcase

Assessment is formative and portfolio-based rather than test-based, tracked across both sessions using a simple facilitator checklist per group:

- **Session 1 evidence:** completed bias-reflection worksheet, a trained model exported with a working shareable link, and at least one documented before/after retraining improvement (screenshot or sticky note of confidence scores).
- **Session 2 evidence:** a running app on their laptop that correctly connects to their own model (not a placeholder), at least one personalised edit (their own slide text, waste tips, or species facts), and one documented tuning change (a threshold or timing value they changed and why).

The Session 2 showcase runs as a "science fair walk": each group stays at their laptop while the rest of the class and facilitator circulate in two rounds of about 5 minutes each, watching a live demo and giving one written piece of feedback on a sticky note (what worked well / one suggestion). The facilitator scores each group live on a simple 3-point scale (Working reliably / Working with minor issues / Needs more time) rather than a numeric grade, and notes one strength and one growth area per group for the parent-facing summary.

"Done" looks like: the webcam feed is live, predictions update continuously with sensible confidence scores, the app responds correctly to at least two of the group's trained classes during a live demo in front of an audience, and the presenting student can explain in their own words what the model was trained to recognise and how the code reacts to it.

Extension & Home Challenges

- **Easy:** Add a fourth class to your Session 1 model (e.g. a new gesture, a new waste category, or a fourth plant species), retrain, export a new link, and update your Session 2 app's code (slides array or `binInfo` object) to use it.
- **Medium:** Add spoken feedback using the browser's built-in Web Speech API so the app speaks its prediction aloud (e.g. `const utter = new SpeechSynthesisUtterance("Recyclable! Use the blue bin."); speechSynthesis.speak(utter);` called inside `showAdvice()`), or add a downloadable CSV log of every scan with a "Download History" button using a Blob and a temporary link element.

- **Hard:** Deploy the finished app online (e.g. GitHub Pages or Netlify's free drag-and-drop deploy) so it works from a shareable public link instead of only on the school laptop, and/or combine two models in one page (for example, a gesture classifier that controls navigation AND a waste classifier shown on the same slide) by loading two separate `ml5.imageClassifier` instances against two different model URLs and merging their results in your app logic.